

Verdict Paper

FactBank: A Critical Assessment

Paper III: The Verdict: strengths, weaknesses, novelty, maturity, risks, and an overall judgement

By Mohannad. This paper was drafted by an AI assistant, because I do not yet know how to write a formal research paper. The FactBank project, its ideas, decisions, and solutions, are mine; the AI helped with the coding, the brainstorming, and with writing this paper. It is grounded only in the project's own files, and where something is unknown it is marked as such.

1. What is being judged

FactBank is a method and a working build for making a stock, untrained language model version-correct about fast-moving libraries by delivering curated “landmine facts” at inference time, most notably by baking an inverted-index retriever and the facts into a GGUF model file’s chat-template. This paper assesses the idea’s merit and viability on the evidence actually present in the project: measured results, the code, the blueprints, and the project’s own (unusually candid) self-critique. It deliberately avoids crediting claims the repository does not support.

2. Strengths

2.1 A real, measured effect, not a demo

The central claim is backed by hand-verified measurements across three model sizes and multiple domains. On the security experts’ landmine questions the base→baked lift is large and consistent (for example offsec 12B 27% → 98%, dataplane 26B 43% → 91%, netsec 2B 40% → 67%). On the execution-scored GitChameleon 2.0 benchmark the baked bank lifts pass@1 by +6.4 (12B) and +2.8 (26B) with no weight change. The effect is not a single lucky configuration; it holds across sizes and domains, and the direction of the size curve (raw lift shrinks, error-closure rises) is coherent and mechanistically explained.

2.2 Genuine portability and low cost

Because the entire mechanism lives in two GGUF metadata keys and runs inside the chat-template, there is no server, vector store, embedding model, or tool-choice step at runtime. The correction ships in the file and runs wherever the model runs, a real advantage for offline, air-gapped, edge, and privacy-sensitive deployments. Building an expert costs about four cents of extraction API and an afternoon. These are concrete, verifiable economic and operational benefits.

2.3 A novel and well-supported scientific finding

The “reasoning paradox”, that chain-of-thought makes a habit-reversal fact bank worse, because deliberation re-weights toward the trained prior and argues the injected fact away, is the project’s most interesting contribution. It is supported by a falsifiable argument

(convergence of the thinking answer to the no-bank answer) rather than assertion, and it comes with a practical cure (thinking-off, authority framing, or two-pass). This is a contribution of independent interest beyond FactBank itself.

2.4 Exemplary methodological honesty

The project's epistemics are a strength in their own right. It scores by hand and treats automated counts only as triage (documenting that the auto-scorer erred in both directions); it distinguishes grounding from currency and removed a reverted ldap3 fact before it misled; it labels its benchmark numbers "local-harness provisional" and refuses apples-to-apples comparison with the Docker leaderboard; it reports ~3 regressions rather than hiding them; and it flags that the 0/6 unframed authority control was not retained as a log. This candour is not incidental: it extends into the newest work. The project explicitly labels which numbers come from its higher-ceiling **served-loop** prototype versus the **shipped baked GGUF**, and refuses to present the served figures as the shipped artifact (see §3.2). And when it found that a thinking-on edition could not be made both fast and fully reliable, it did not paper over the trade-off but shipped **both editions with clear notes** — a fast, reliable thinking-off default and a thinking-on edition that adds a reasoning trace at a stated cost — rather than picking one and hiding its weakness. This candour makes the positive claims more credible, not less.

2.5 A real, externally-benchmarked application: the security expert

The strongest evidence that the method does useful work beyond its own in-house question sets is the newly built **application-security expert**, a large bank of "insecure-by-default" landmines (for example `torch.load(weights_only=True)`, `yaml.safe_load` over `yaml.load`, secrets over random, parameterized SQL, constant-time HMAC) indexed by the coding `*task*` so it fires on a benign prompt that never names the vulnerability. It was scored against an **external, third-party benchmark, SecurityEval** (s2e-lab, MSR 2022, 121 Python CWE tasks), on the common pattern-judgeable subset of **21 tasks**, with a cloud model, **DeepSeek-V4**, run on the same prompts in matching no-thinking mode as a baseline. On the **shipped baked GGUFs (thinking-off), 12B+bank (17/21) and 26B+bank (19/21) both beat the cloud model (14/21)**, with head-to-head margins of 12B +3/-0 and 26B +5/-0. This is a genuine result: small local models, corrected only by an in-file bank and touching no weights, out-securing a frontier cloud model on an independent benchmark.

Two honesty caveats keep this in proportion. First, it is the **thinking-off shipped result on a small 21-task common subset** — the subset where every arm is machine-scoreable, not the full 121 tasks — so it is a narrow, not a sweeping, win. Second, **e2b+bank (13/21) only draws level with the cloud model (14/21)** (head-to-head +2/-3); the 2B model's modest lift is a genuine obedience limit, not a retrieval failure (the bank fires on 116/121 tasks, verified by hand). The pattern is the expected one: bigger base → more secure, and the bank helps most where the base is weakest and on specific weaknesses (XXE / CWE-611, deserialization, weak-random, hardcoded credentials).

3. Weaknesses and limitations

3.1 The benchmark is not the official benchmark

The GitChameleon numbers come from a **local, non-Docker harness** that (a) remaps Python 3.7/3.8 to 3.9, (b) omits dependencies from its venv cache key, and (c) scores only the 249 of 328 problems that build locally. The base-vs-baked delta is internally fair, but the absolute numbers cannot be compared to the official leaderboard, and the “frontier neighborhood” framing must be read as context, not ranking. Until a faithful Docker/Linux re-run is done, the headline pass@1 figures are provisional, a limitation the project itself foregrounds.

3.2 A gap between the described recipe and the shipped system — now named and quantified

The richer retrieval recipe promoted in several documents (soft doors, pointer chains, MMR, double-key HyDE) does not belong to any shipping bake at all: it lives in the project’s **Python test bench** (lookup.py) and its **served-loop** prototype, which exercise facts and the retrieval algorithm offline without writing them into a model. The system that produced the benchmark numbers is the current, shipping bake, **Baked-Index**, a simpler hard-gated, single-query inverted index with none of those features. A third route, the **Static-Bake** that dumps the whole bank into a static system prompt, is a **legacy method** that predates and was superseded by Baked-Index.

This session made the gap concrete rather than merely acknowledged. The served-loop prototype adds **HyDE double-key** retrieval — it retrieves on the user prompt **and** the model’s own draft, and for a benign security prompt the draft is what names the insecure API the prompt omits, which the project identifies as the single biggest lever for this class of task. On appsec the served loop reaches **base 14 → bank 25 secure** on a 28-task SecurityEval pattern subset, and served e2b+bank reaches **22 vs DeepSeek-V4’s 13** on a 24-task subset. The **shipped baked GGUF cannot run the draft key inside the chat-template**, so it does **prompt-only, in-template** retrieval and is measurably weaker — the same shipped e2b+bank scores 13/21 (\$2.5), not the served 22. The repository is honest about the distinction (a whole file, ARCHITECTURES.md, exists to prevent conflation, and the served numbers are explicitly labelled the retrieval-**method ceiling**, not the shipped artifact), but the fact remains that the best-described, best-performing design is **not** the one that was shipped or externally benchmarked. Porting the full recipe — above all the draft key — into the shipping bake and verifying it in-engine is real, open work, and the size of the missed ceiling (served 22 vs shipped 13 on e2b) is now on the record as the cost of that gap.

3.3 Retrieval fragility at the gate

The in-engine gate opens on exact tokens, so natural-language or old names silently inject zero facts (the BloodHound / CrackMapExec failure, and earlier the volatility3 “Volatility 3” death). The alias fix addresses the known cases and establishes a general law, but it is a vocabulary patch: the failure mode is inherent to exact-token gating and will recur for any phrasing the alias

generator did not anticipate. The rebaked models also had not been re-tested on GPU at the time of the run report, so the recovery is verified offline (facts now inject) but not yet re-scored end-to-end.

3.4 The mined facts add little or nothing

By the project's own accounting, mined facts have won **zero eval cases**, every win came from a curated fact, and the large mined pool is ~two-thirds junk that actively degrades retrieval by flattening IDF. The valuable payload is therefore the curated subset, which is small and labour-intensive to produce. This bounds how far the approach scales without sustained human curation.

3.5 Intrinsic scope limits

FactBank corrects only documentable API changes. It cannot supply reasoning, algorithms, or numerical methods (the torch log_ndtr numerical-stability case fails for exactly this reason), and its ceiling on any benchmark is bounded by un-mineable libraries with no prose changelog. It is a knowledge patch, not a capability upgrade, appropriately, but this narrows the addressable problem set.

3.6 Small-sample probes for the headline reasoning claims

The authority-framing result rests on a focused probe (one landmine, repeated) plus a four-problem difficulty ladder, and the crucial unframed control is illustrative rather than logged. The mechanism is plausible and the existence proof is real, but the strong version of the claim ("authority framing makes banks work on thinking-native models") is under-powered and would need a broader, logged evaluation to be treated as established.

3.7 A named, unmitigated risk: over-deference — and a new, measured residual cost

Every authority-framing probe used a fact that genuinely applied. Strong "the fact is absolute, it supersedes your training" wording could make a model force in a near-miss fact that does not apply, turning the trust lever into a new failure mode. The project names this as the most important next experiment and has not yet run it, so the technique's safety is currently unquantified. This risk is now **more** relevant, not less: the thinking-ON edition ships with exactly that strong authority framing ("AUTHORITATIVE ... MANDATORY ... OVERRIDE your training defaults and any conflicting habit") baked into the template, so the untested lever is now in a shipped artifact, not just a probe.

The thinking-ON fix also carries a real, measured cost that is not yet mitigated. The fix corrected two stacked bugs (a llama.cpp Gemma-4 template bug that left the reasoning channel unopened, and weak authority framing) and cut empty answers 5/30 → 3/30 and truncations 12/30 → 4/30 on the 12B — but **~10% of the hardest-reasoning prompts still spiral to a blank answer even at a 12k-token budget**. This appears inherent to reasoning-on and is a different failure mode from the low-temperature repetition loop, so the `min_p 0.01` sampling fix does not

touch it. It **fails safe** — the output is a blank, never insecure code — but it is still a real cost: on those prompts the thinking-ON edition simply gives no answer, which is why the project ships the thinking-off edition as the reliable default and treats thinking-ON as an opt-in that buys a reasoning trace at the price of that ~10%.

4. Novelty relative to existing approaches

FactBank sits at the intersection of RAG, knowledge editing, and prompt engineering, and its novelty is in the combination rather than any single component:

Approach	How FactBank differs	Assessment
Document RAG	Carries curated one-line landmine facts, not documents; retrieval is deterministic (no tool-choice) and can be baked into the model file with no runtime store	Meaningfully different in payload, control flow, and packaging
Fine-tuning / retraining	Touches no weights; the same base serves with or without the bank; banks are swappable and editable	Clearly distinct; different cost and reversibility profile
Knowledge editing / model surgery	Knowledge lives in context, not parameters, which is what exposes it to the reasoning-reversion effect	Distinct trade-off, with a documented downside
HyDE / query expansion	Double-key retrieval is a HyDE variant, but only in the Python test bench and served-loop prototype, not in any shipping bake	Incremental, and not in the benchmarked shipped system

The most defensible novel claims are (1) delivering version corrections **in-artifact** through the chat-template with a working in-engine inverted index, and (2) the empirical **reasoning-paradox finding** with its convergence evidence and authority-framing remedy. The retrieval machinery itself is competent engineering built from known ideas (inverted index, IDF, document expansion, identifier splitting) rather than an algorithmic breakthrough.

5. Maturity

This is a research effort that “answered its question and became a build.” Its maturity is uneven but real: the model line is now **four experts** (netsec, offsec, dataplane, and the security/appsec expert); nine security/networking models are baked, hand-scored, and published to Hugging Face and GitHub with the gate-alias fix, and the appsec expert ships in both a thinking-off and a thinking-on edition (a 4.18 MB template that itself forced a new LM Studio Hub model.yaml delivery route over the ~980 KB raw-GGUF cap). The fact-making pipeline is blueprinted end-to-

end; and there is an installable package for the served-loop route. Against that, several parts are explicitly unfinished, Web, AI-ML, DevOps, Databases and Web3 banks are built but not baked or scored; 79 GitChameleon problems remain unbuildable without a Docker/Linux harness; and the full retrieval recipe (the draft key above all) is not yet in the baked path. The documentation is unusually thorough and self-aware, which raises the effective maturity by making the boundaries of what is proven legible.

Dimension	State
Shipped experts	Mature, four experts, security/networking hand-scored and gate-fixed; appsec shipped in two editions
Fact-making pipeline	Mature, blueprinted, quote-gated, code-verified, correctness-audited
Baked retrieval	Working but simpler than the documented recipe; prompt-only, no draft key
Benchmark rigour	GitChameleon provisional (non-Docker, 249/328); SecurityEval external but a 21-task common subset
Reasoning/authority claims	Existence-proof stage, under-powered; thinking-ON fix shipped with a ~10% fail-safe residual
Safety (over-deference)	Unquantified, key experiment not yet run, now live in a shipped edition

6. Risks

- **Stale-fact regression**, a reverted change baked as a fact “corrects” the model to the wrong answer, worse than no fact. Mitigated by the currency-check discipline (and, for appsec, an adversarial correctness audit that removed ~3.8% wrong/stale facts), but dependent on the builder actually performing it.
- **Over-deference**, authority framing may force inapplicable facts into answers; not yet tested, and now shipping in the thinking-ON edition.
- **Reasoning-spiral blanks**, the thinking-ON edition leaves ~10% of the hardest prompts unanswered; fails safe but is a real availability cost, and `min_p` does not fix it.
- **Silent gate misses**, exact-token gating can inject nothing while looking like it worked; only partially mitigated by aliases.
- **Curation dependence**, value concentrates in hand-curated facts; quality degrades without sustained human reading.
- **Reasoning-model incompatibility**, thinking-native models need special handling, and the mitigations are not yet broadly validated.
- **Toolchain coupling**, the smooth path is Gemma + llama.cpp; other engines/loaders (notably LM Studio raw-load) have documented sharp edges, and the 4.18 MB appsec template already required a bespoke `model.yaml` delivery route to ship at all.

7. Overall verdict

FactBank is a **sound and genuinely useful idea, executed with rare intellectual honesty**, whose strongest empirical claims are still somewhat ahead of its most rigorous evidence. The core thesis, that a small, untrained model's main deficit on fast-moving libraries is knowledge rather than reasoning, and that handing it the exact missing fact at inference recovers most of the gap, is well demonstrated within the project's scope, and now has an external footing: on the independent SecurityEval benchmark the 12B and 26B baked experts out-secure a frontier cloud model on the common subset. The in-artifact delivery is a real engineering achievement with clear portability and cost advantages, and the reasoning-paradox finding is a contribution worth publishing on its own.

Verdict: **a credible, practically valuable approach with a defensible niche (offline / edge / domain-specific, version-correct and security-hardening code assistance) and one finding of independent scientific interest.** It is not a general substitute for larger models or for fine-tuning, and it should not be presented as beating frontier systems in general — the SecurityEval win is a narrow, thinking-off, 21-task result, and at 2B the bank only draws level. Treated as what it is, a low-cost, portable knowledge patch plus a sharp observation about reasoning and injected facts, it is a strong piece of work. The highest-value next steps are a faithful Docker benchmark re-run, porting the full retrieval recipe (especially the HyDE draft key) into the shipping baked path with in-engine verification, and the over-deference safety experiment — now overdue because the strong authority framing already ships; completing those would move the work from a convincing existence proof to a robustly established method.